CSA3714-SOFTWARE TESTING

WHITE-BOX TESTING – JUNIT LAB PROGRAMS

16. Demonstrate the working of JUnit to check whether a string is an anagram of another string and use assert statements to validate the output using white-box testing.

AIM

To check whether two strings are anagrams and validate the result using JUnit 5.

PROCEDURE

1. Create an implementation class and a JUnit test class.
2. Remove spaces and convert both strings to lowercase.
3. Sort the character arrays and compare them.
4. Use assertions to test anagram and non-anagram inputs.

PROGRAM

Implementation file:

```
package com.softwaretesting;

import java.util.Arrays;

public class Exp16 {

    public boolean isAnagram(String str1, String str2) {

        if (str1 == null || str2 == null) {
            return false;
        }

        str1 = str1.replaceAll("\\s", "").toLowerCase();
        str2 = str2.replaceAll("\\s", "").toLowerCase();

        if (str1.length() != str2.length()) {
            return false;
        }
```

1923136

```
        char[] a = str1.toCharArray();
        char[] b = str2.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        return Arrays.equals(a, b);
    }
    public static void main(String[] args) {
        Exp16 obj = new Exp16();
        System.out.println("ANAGRAM CHECKING");
        System.out.println("-----------------------------");
        System.out.println("listen, silent : " +
                obj.isAnagram("listen", "silent"));
        System.out.println("hello, world : " +
                obj.isAnagram("hello", "world"));
        System.out.println("Dormitory, Dirty room : " +
                obj.isAnagram("Dormitory", "Dirty room"));
        System.out.println("apple, apples : " +
                obj.isAnagram("apple", "apples"));
    }
}
```

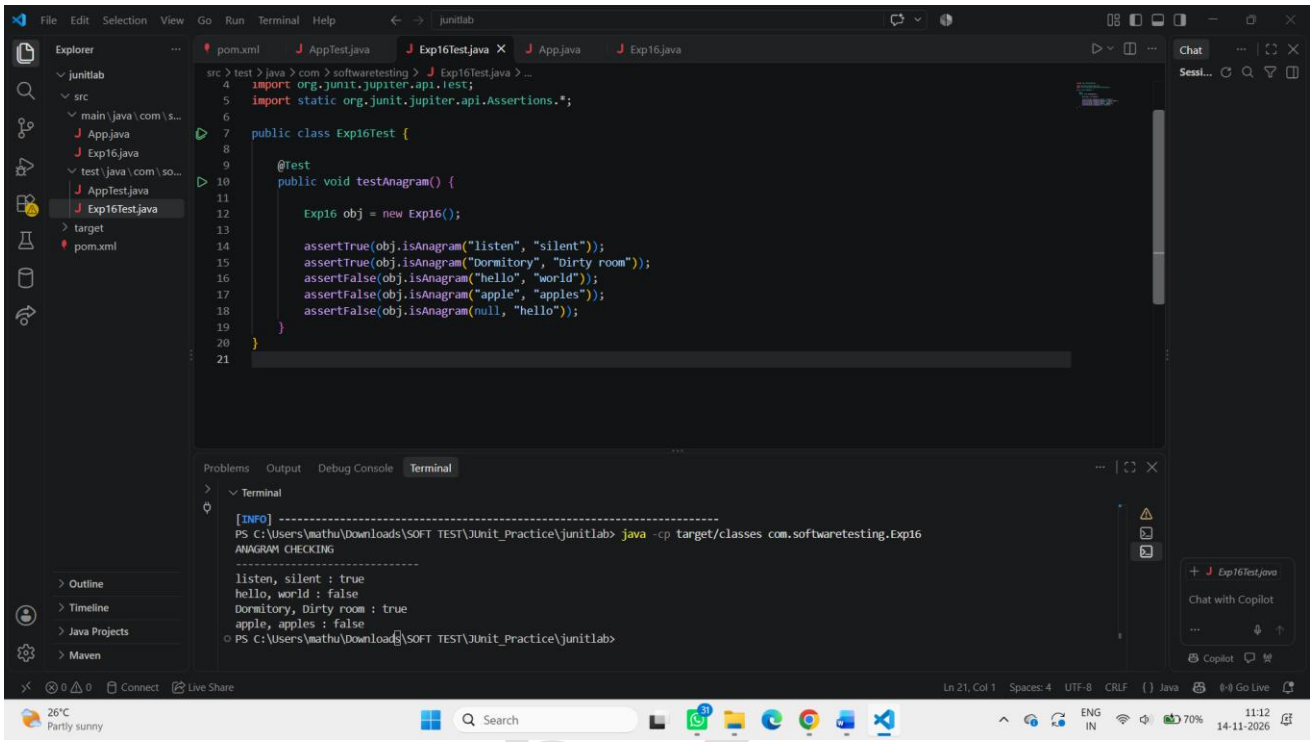
JUnit test file:

```
package com.softwaretesting;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class Exp16Test {
    @Test
    public void testAnagram() {
        Exp16 obj = new Exp16();
        assertTrue(obj.isAnagram("listen", "silent"));
        assertTrue(obj.isAnagram("Dormitory", "Dirty room"));
        assertFalse(obj.isAnagram("hello", "world"));
        assertFalse(obj.isAnagram("apple", "apples"));
```

192311363

```
        assertFalse(obj.isAnagram(null, "hello"));
    }
}
```

OUTPUT



RESULT

Thus, the anagram-checking function was tested successfully using JUnit assertions.

17. Write a white-box JUnit test case to find the maximum of three numbers and validate the output using appropriate assert methods.

AIM

To find the maximum of three integers and verify the result using JUnit assertions.

PROCEDURE

1. Create a function to compare three integer values.
2. Use if-else conditions to identify the greatest value.
3. Test cases where each argument is the maximum and where values are equal.
4. Run the JUnit test using Maven.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp17 {

   public int findMaximum(int a, int b, int c) {

      if (a >= b && a >= c) {
         return a;
      } else if (b >= a && b >= c) {
         return b;
      } else {
         return c;
      }
   }

   public static void main(String[] args) {

      Exp17 obj = new Exp17();

      System.out.println("MAXIMUM OF THREE NUMBERS");
      System.out.println("------------------------------");
```

1931363

```
        System.out.println("10, 20, 30 : " + obj.findMaximum(10, 20, 30));
        System.out.println("50, 25, 15 : " + obj.findMaximum(50, 25, 15));
        System.out.println("12, 40, 18 : " + obj.findMaximum(12, 40, 18));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp17Test {

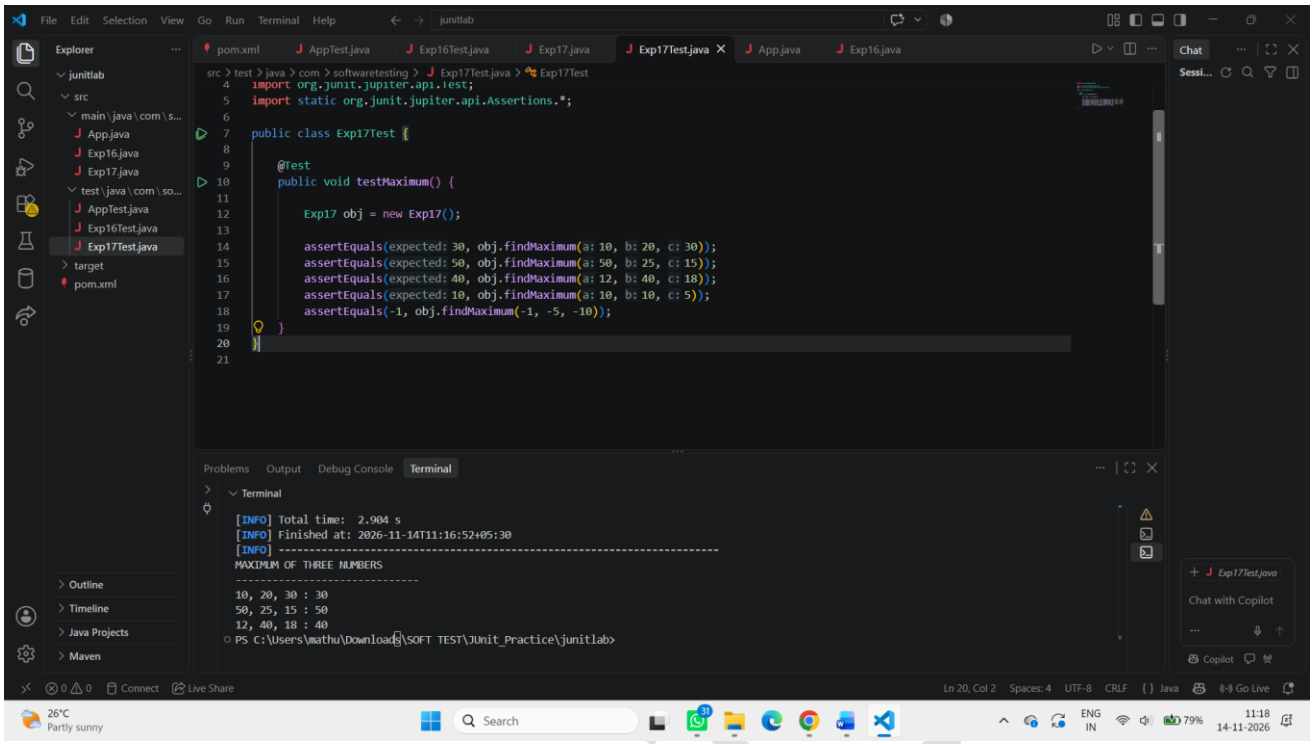
    @Test
    public void testMaximum() {

        Exp17 obj = new Exp17();

        assertEquals(30, obj.findMaximum(10, 20, 30));
        assertEquals(50, obj.findMaximum(50, 25, 15));
        assertEquals(40, obj.findMaximum(12, 40, 18));
        assertEquals(10, obj.findMaximum(10, 10, 5));
        assertEquals(-1, obj.findMaximum(-1, -5, -10));
    }
}
```

1923136 3

OUTPUT



RESULT

Thus, the maximum of three numbers was found and validated successfully using JUnit.

192319

18. Write a JUnit test case to check whether a number is even or odd, and validate the result using white-box testing techniques.

AIM

To determine whether an integer is even or odd and verify the result using JUnit.

PROCEDURE

1. Create a function that checks the remainder after division by two.
2. Return true for even numbers and false for odd numbers.
3. Test even, odd, zero, and negative values.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp18 {

    public String checkEvenOdd(int number) {

        if (number % 2 == 0) {
            return "Even";
        } else {
            return "Odd";
        }
    }

    public static void main(String[] args) {

        Exp18 obj = new Exp18();

        System.out.println("EVEN OR ODD CHECKING");
        System.out.println("-----------------------------");

        System.out.println("10 : " + obj.checkEvenOdd(10));
        System.out.println("7 : " + obj.checkEvenOdd(7));
        System.out.println("0 : " + obj.checkEvenOdd(0));
        System.out.println("-5 : " + obj.checkEvenOdd(-5));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp18Test {
```

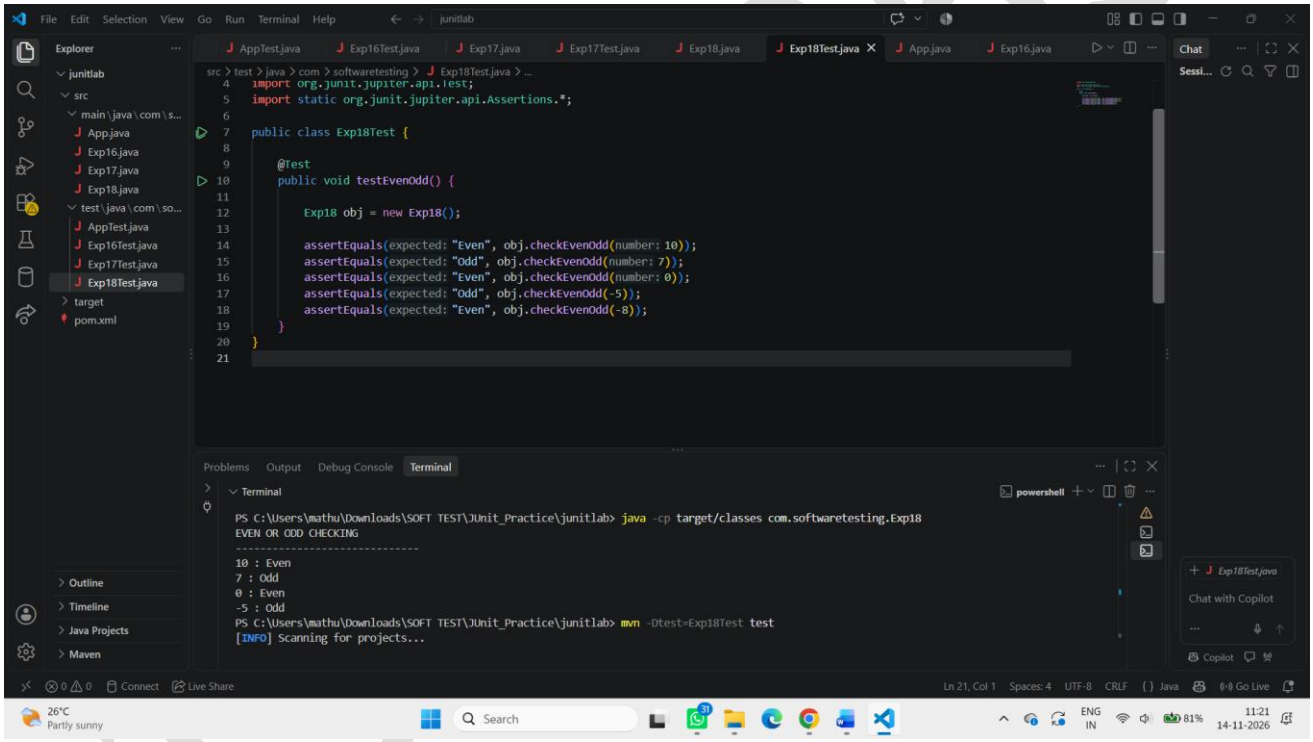
1923136

```
    @Test
    public void testEvenOdd() {

        Exp18 obj = new Exp18();

        assertEquals("Even", obj.checkEvenOdd(10));
        assertEquals("Odd", obj.checkEvenOdd(7));
        assertEquals("Even", obj.checkEvenOdd(0));
        assertEquals("Odd", obj.checkEvenOdd(-5));
        assertEquals("Even", obj.checkEvenOdd(-8));
    }
}
```

OUTPUT



RESULT

Thus, even and odd numbers were identified and verified successfully using JUnit.

19. Write a program using functions to calculate compound interest. Verify the output values using white-box testing and JUnit assertions.

AIM

To calculate compound interest using a Java function and validate the result using JUnit.

PROCEDURE

1. Create a function accepting principal, annual rate, compounding frequency, and years.
2. Calculate amount using A = P(1 + r/n)^(nt).
3. Subtract principal to obtain compound interest.
4. Test normal and zero-interest cases using JUnit.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp19 {

    public double calculateCompoundInterest(double principal, double rate, double time) {

        double amount = principal * Math.pow((1 + rate / 100), time);

        double compoundInterest = amount - principal;

        return compoundInterest;
    }

    public static void main(String[] args) {

        Exp19 obj = new Exp19();

        System.out.println("COMPOUND INTEREST CALCULATION");
        System.out.println("-------------------------------");

        System.out.println("Principal: 1000, Rate: 10%, Time: 2 years");
```

1923133

```
        System.out.println("Compound Interest: " +
                obj.calculateCompoundInterest(1000, 10, 2));

        System.out.println("Principal: 1500, Rate: 5%, Time: 2 years");
        System.out.println("Compound Interest: " +
                obj.calculateCompoundInterest(1500, 5, 2));

        System.out.println("Principal: 2000, Rate: 8%, Time: 1 year");
        System.out.println("Compound Interest: " +
                obj.calculateCompoundInterest(2000, 8, 1));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp19Test {

    @Test
    public void testCompoundInterest() {

        Exp19 obj = new Exp19();

        assertEquals(210.0,
                obj.calculateCompoundInterest(1000, 10, 2), 0.001);

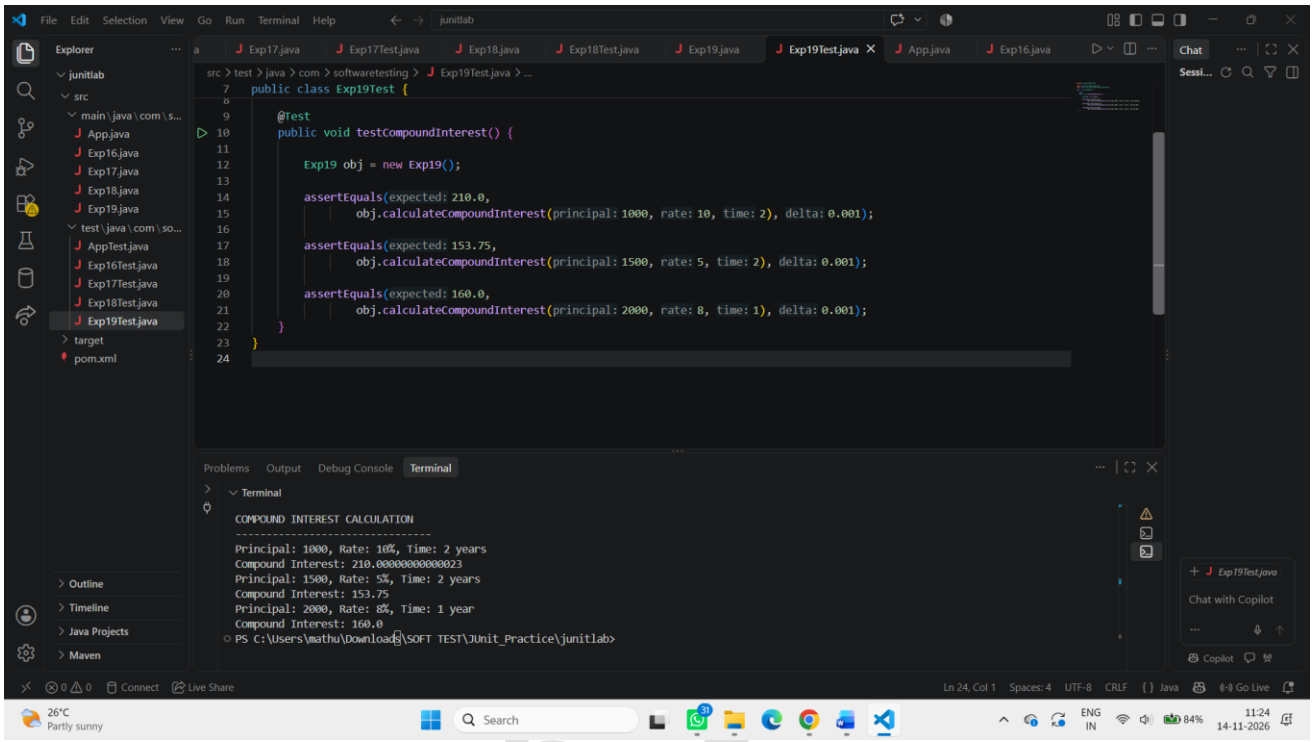
        assertEquals(153.75,
                obj.calculateCompoundInterest(1500, 5, 2), 0.001);

        assertEquals(160.0,
```

1923133

```
            obj.calculateCompoundInterest(2000, 8, 1), 0.001);
    }
}
```

OUTPUT



RESULT

Thus, compound interest was calculated and verified successfully using JUnit assertions.

192

20. Write a JUnit program to count the number of vowels and consonants in a given string and validate the output using assert statements.

AIM

To count vowels and consonants in a string and verify both counts using JUnit.

PROCEDURE

1. Traverse each character in the input string.
2. Ignore spaces, digits, and symbols.
3. Count a, e, i, o, u as vowels and other letters as consonants.
4. Return both counts and validate them using assertions.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp20 {

   public int countVowels(String str) {

      int count = 0;

      for (int i = 0; i < str.length(); i++) {
         char ch = Character.toLowerCase(str.charAt(i));

         if (ch == 'a' || ch == 'e' || ch == 'i' ||
            ch == 'o' || ch == 'u') {
            count++;
         }
      }

      return count;
   }

   public int countConsonants(String str) {
```

1923136

```
        int count = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = Character.toLowerCase(str.charAt(i));

            if (ch >= 'a' && ch <= 'z' &&
                !(ch == 'a' || ch == 'e' || ch == 'i' ||
                  ch == 'o' || ch == 'u')) {
                count++;
            }
        }

        return count;
    }

    public static void main(String[] args) {

        Exp20 obj = new Exp20();

        String str = "Hello World";

        System.out.println("VOWELS AND CONSONANTS COUNT");
        System.out.println("----------------------------");
        System.out.println("String: " + str);
        System.out.println("Vowels: " + obj.countVowels(str));
        System.out.println("Consonants: " + obj.countConsonants(str));
    }
}
```

JUnit test file:

```
package com.softwaretesting;
```

1923136

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp20Test {

    @Test
    public void testVowelsAndConsonants() {

        Exp20 obj = new Exp20();

        assertEquals(3, obj.countVowels("Hello World"));
        assertEquals(7, obj.countConsonants("Hello World"));

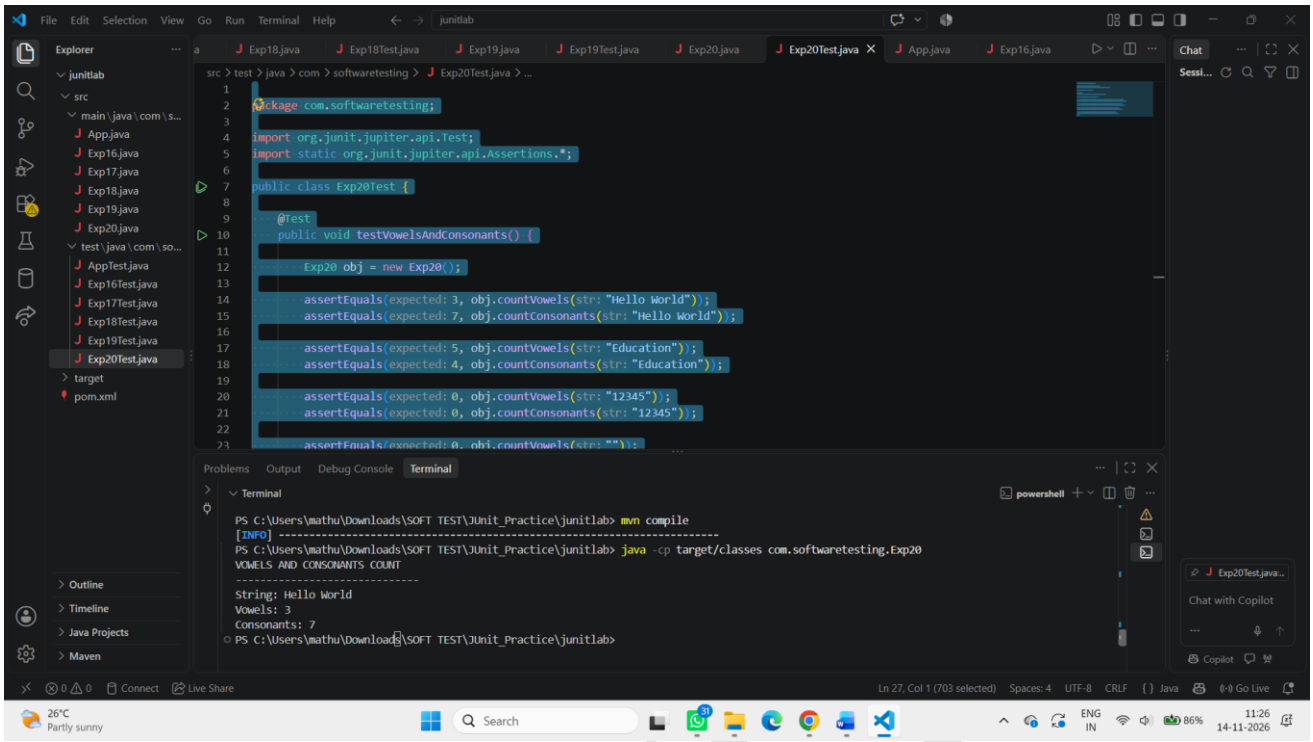
        assertEquals(5, obj.countVowels("Education"));
        assertEquals(4, obj.countConsonants("Education"));

        assertEquals(0, obj.countVowels("12345"));
        assertEquals(0, obj.countConsonants("12345"));

        assertEquals(0, obj.countVowels(""));
        assertEquals(0, obj.countConsonants(""));
    }
}
```

1931363

OUTPUT



RESULT

Thus, the vowel and consonant counts were calculated and validated successfully using JUnit.

192115

21. Write a program to check whether a given number is a prime number. Verify the output using white-box testing with JUnit assertions.

AIM

To determine whether a number is prime and validate the result using JUnit.

PROCEDURE

1. Return false for numbers less than two.
2. Check divisibility from 2 up to the square root of the number.
3. Return false if a divisor is found; otherwise return true.
4. Test prime, composite, zero, one, and negative inputs.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp21 {

    public boolean isPrime(int number) {

        if (number <= 1) {
            return false;
        }

        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;
            }
        }

        return true;
    }

    public static void main(String[] args) {
```

1923136

```
        Exp21 obj = new Exp21();

        System.out.println("PRIME NUMBER CHECKING");
        System.out.println("-----------------------------");

        System.out.println("7 : " + (obj.isPrime(7) ? "Prime" : "Not Prime"));
        System.out.println("10 : " + (obj.isPrime(10) ? "Prime" : "Not Prime"));
        System.out.println("1 : " + (obj.isPrime(1) ? "Prime" : "Not Prime"));
        System.out.println("13 : " + (obj.isPrime(13) ? "Prime" : "Not Prime"));
        System.out.println("-5 : " + (obj.isPrime(-5) ? "Prime" : "Not Prime"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp21Test {

    @Test
    public void testPrimeNumber() {

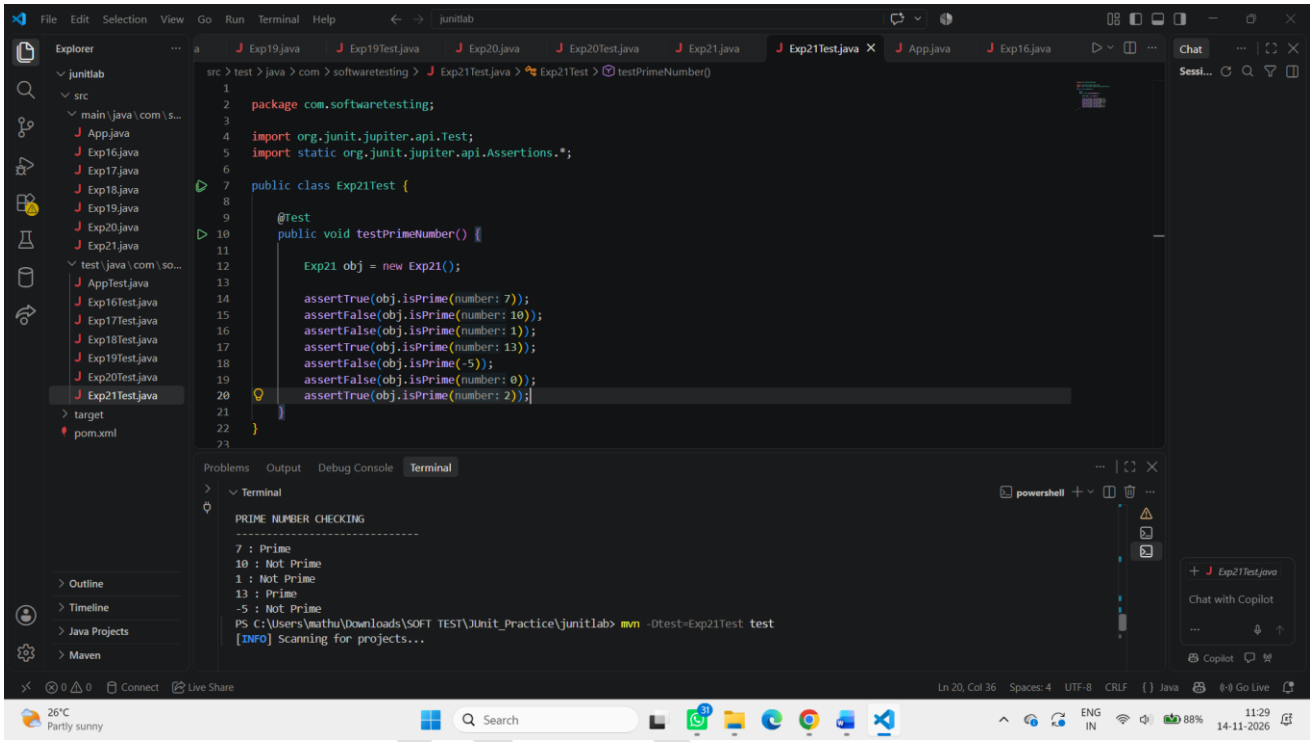
        Exp21 obj = new Exp21();

        assertTrue(obj.isPrime(7));
        assertFalse(obj.isPrime(10));
        assertFalse(obj.isPrime(1));
        assertTrue(obj.isPrime(13));
        assertFalse(obj.isPrime(-5));
        assertFalse(obj.isPrime(0));
        assertTrue(obj.isPrime(2));
```

1923136

```
    }
}
```

OUTPUT



RESULT

Thus, prime and non-prime numbers were identified and verified successfully using JUnit.

1923

22. Write a JUnit test case to reverse a number and validate the output using white-box testing techniques.

AIM

To reverse an integer and validate the reversed value using JUnit.

PROCEDURE

1. Extract the last digit using the modulus operator.
2. Append each digit to the reversed number.
3. Remove the last digit using integer division.
4. Test positive, negative, zero, and single-digit values.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp22 {

   public int reverseNumber(int number) {

      int reverse = 0;

      while (number != 0) {
         int digit = number % 10;
         reverse = reverse * 10 + digit;
         number = number / 10;
      }

      return reverse;
   }

   public static void main(String[] args) {

      Exp22 obj = new Exp22();
```

1923136

```
        System.out.println("REVERSE OF A NUMBER");
        System.out.println("-----------------------------");

        System.out.println("1234 : " + obj.reverseNumber(1234));
        System.out.println("567 : " + obj.reverseNumber(567));
        System.out.println("100 : " + obj.reverseNumber(100));
        System.out.println("0 : " + obj.reverseNumber(0));
        System.out.println("-123 : " + obj.reverseNumber(-123));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp22Test {

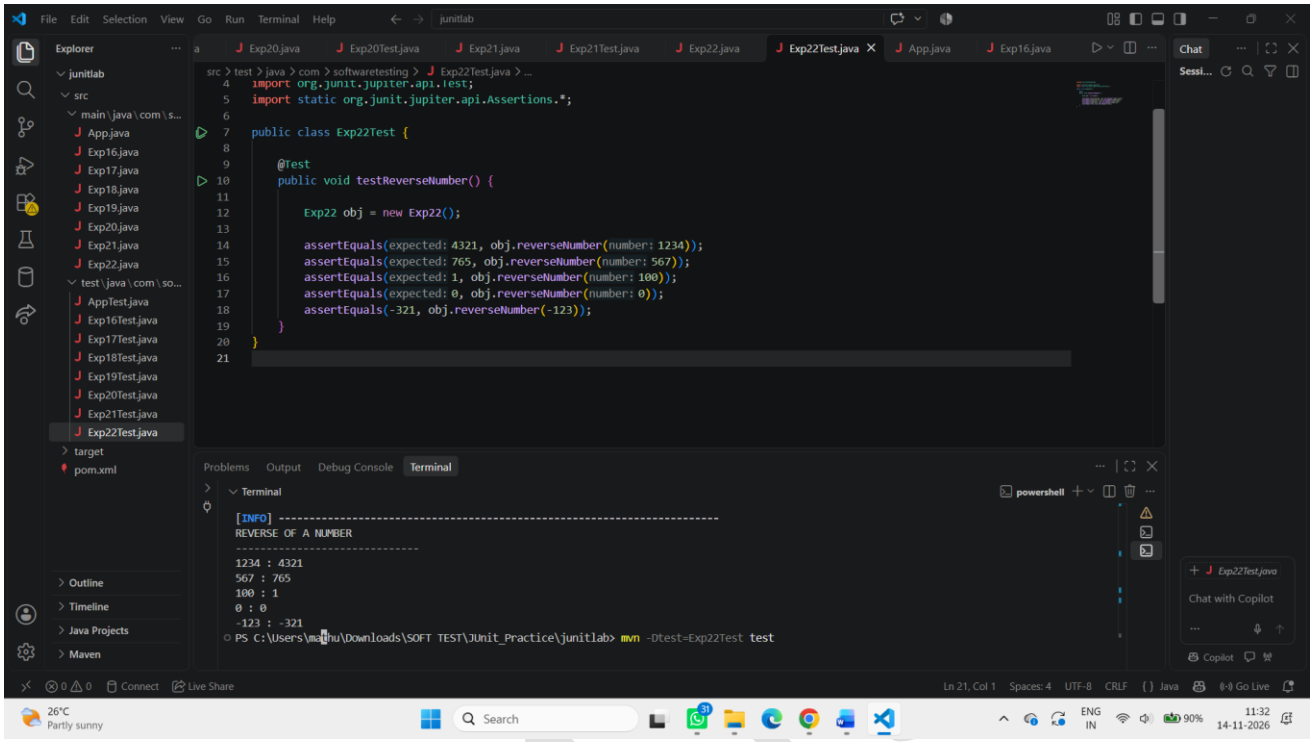
    @Test
    public void testReverseNumber() {

        Exp22 obj = new Exp22();

        assertEquals(4321, obj.reverseNumber(1234));
        assertEquals(765, obj.reverseNumber(567));
        assertEquals(1, obj.reverseNumber(100));
        assertEquals(0, obj.reverseNumber(0));
        assertEquals(-321, obj.reverseNumber(-123));
    }
}
```

1923163

OUTPUT



RESULT

Thus, the given number was reversed and validated successfully using JUnit.

192211

23. Write a program to find the sum of digits of a given number and verify the output using white-box testing.

AIM

To calculate the sum of the digits of an integer and verify the result using JUnit.

PROCEDURE

1. Extract each digit using the modulus operator.
2. Add each extracted digit to the sum.
3. Remove the last digit using integer division.
4. Test normal, zero, and negative inputs.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp23 {

   public int sumOfDigits(int number) {

      int sum = 0;
      number = Math.abs(number);

      while (number != 0) {
         int digit = number % 10;
         sum = sum + digit;
         number = number / 10;
      }

      return sum;
   }

   public static void main(String[] args) {

      Exp23 obj = new Exp23();
```

1931363

```
        System.out.println("SUM OF DIGITS");
        System.out.println("------------------------------");

        System.out.println("1234 : " + obj.sumOfDigits(1234));
        System.out.println("567 : " + obj.sumOfDigits(567));
        System.out.println("100 : " + obj.sumOfDigits(100));
        System.out.println("0 : " + obj.sumOfDigits(0));
        System.out.println("-123 : " + obj.sumOfDigits(-123));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp23Test {

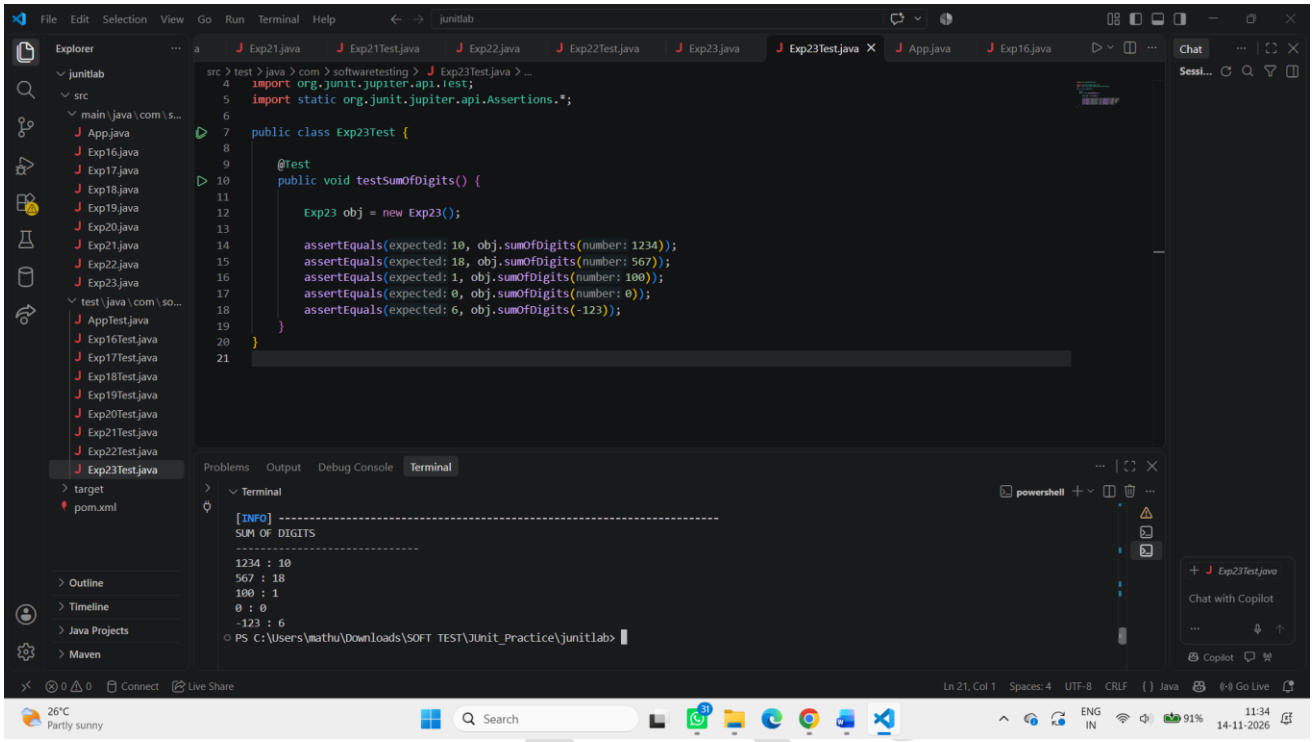
    @Test
    public void testSumOfDigits() {

        Exp23 obj = new Exp23();

        assertEquals(10, obj.sumOfDigits(1234));
        assertEquals(18, obj.sumOfDigits(567));
        assertEquals(1, obj.sumOfDigits(100));
        assertEquals(0, obj.sumOfDigits(0));
        assertEquals(6, obj.sumOfDigits(-123));
    }
}
```

1923133

OUTPUT



RESULT

Thus, the sum of the digits was calculated and verified successfully using JUnit.

192317

24. Write a JUnit program to check whether two strings are equal ignoring case sensitivity, and validate the output using assert statements.

AIM

To compare two strings without considering uppercase and lowercase differences and validate the result using JUnit.

PROCEDURE

1. Create a function using equalsIgnoreCase().
2. Return the boolean comparison result.
3. Test strings that match with different cases and strings that do not match.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp24 {

   public boolean compareStrings(String str1, String str2) {

      if (str1 == null || str2 == null) {
         return str1 == str2;
      }

      return str1.equalsIgnoreCase(str2);
   }

   public static void main(String[] args) {

      Exp24 obj = new Exp24();

      System.out.println("STRING COMPARISON (IGNORE CASE)");
      System.out.println("------------------------------");

      System.out.println("Hello, hello : " +
```

1923163

```
            obj.compareStrings("Hello", "hello"));

        System.out.println("JAVA, java : " +
            obj.compareStrings("JAVA", "java"));

        System.out.println("Apple, Orange : " +
            obj.compareStrings("Apple", "Orange"));

        System.out.println("Test, TEST : " +
            obj.compareStrings("Test", "TEST"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp24Test {

    @Test
    public void testCompareStrings() {

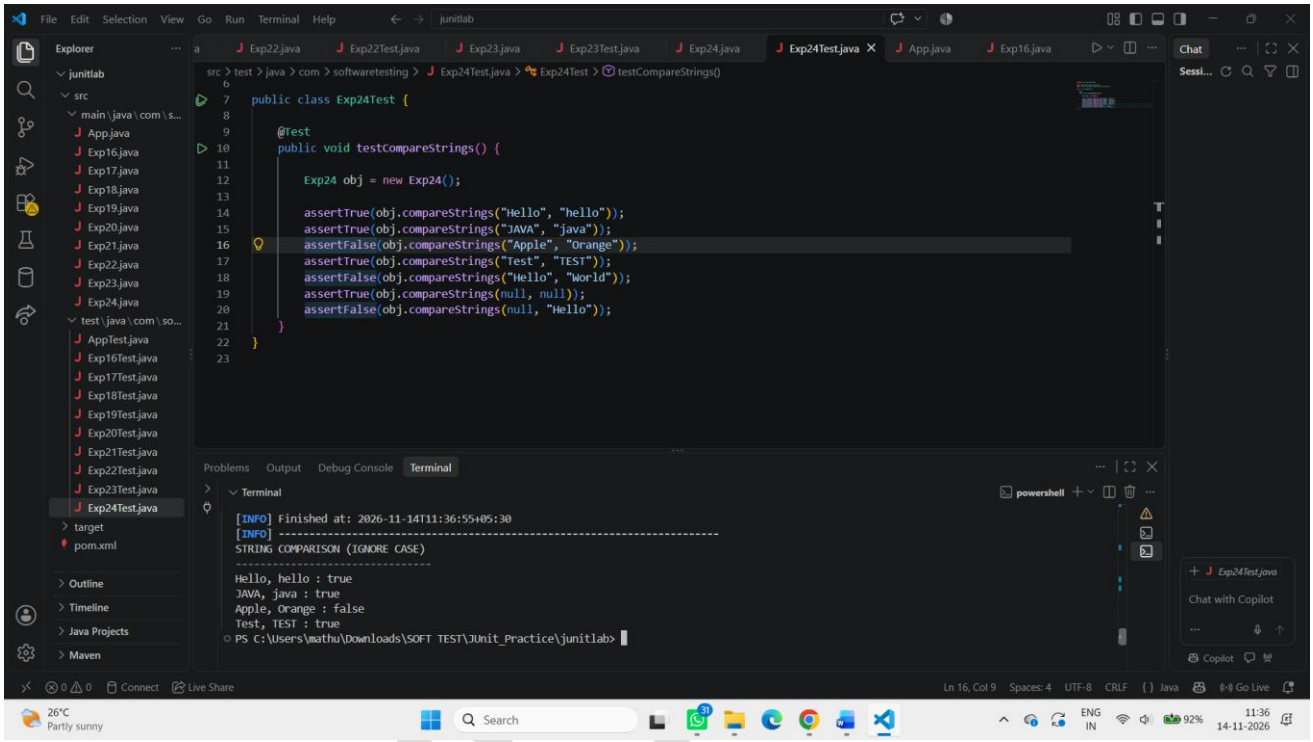
        Exp24 obj = new Exp24();

        assertTrue(obj.compareStrings("Hello", "hello"));
        assertTrue(obj.compareStrings("JAVA", "java"));
        assertFalse(obj.compareStrings("Apple", "Orange"));
        assertTrue(obj.compareStrings("Test", "TEST"));
        assertFalse(obj.compareStrings("Hello", "World"));
        assertTrue(obj.compareStrings(null, null));
        assertFalse(obj.compareStrings(null, "Hello"));
```

1923133

```
    }
}
```

OUTPUT



RESULT

Thus, strings were compared without case sensitivity and validated successfully using JUnit.

1923

25. Write a program using functions to calculate the grade of a student based on marks. Verify the output using white-box testing.

AIM

To calculate a student's grade based on marks and verify each grading condition using JUnit.

PROCEDURE

1. Accept marks as input to a function.
2. Use if-else conditions to assign a grade.
3. Return Invalid for marks outside 0 to 100.
4. Test each grade branch and invalid values.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp25 {

    public String calculateGrade(int marks) {

        if (marks < 0 || marks > 100) {
            return "Invalid Marks";
        } else if (marks >= 90) {
            return "A";
        } else if (marks >= 80) {
            return "B";
        } else if (marks >= 70) {
            return "C";
        } else if (marks >= 60) {
            return "D";
        } else {
            return "Fail";
        }
    }
```

1923136

```
    public static void main(String[] args) {

        Exp25 obj = new Exp25();

        System.out.println("STUDENT GRADE CALCULATION");
        System.out.println("-----------------------------");

        System.out.println("95 : " + obj.calculateGrade(95));
        System.out.println("85 : " + obj.calculateGrade(85));
        System.out.println("75 : " + obj.calculateGrade(75));
        System.out.println("65 : " + obj.calculateGrade(65));
        System.out.println("40 : " + obj.calculateGrade(40));
        System.out.println("105 : " + obj.calculateGrade(105));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp25Test {

    @Test
    public void testCalculateGrade() {

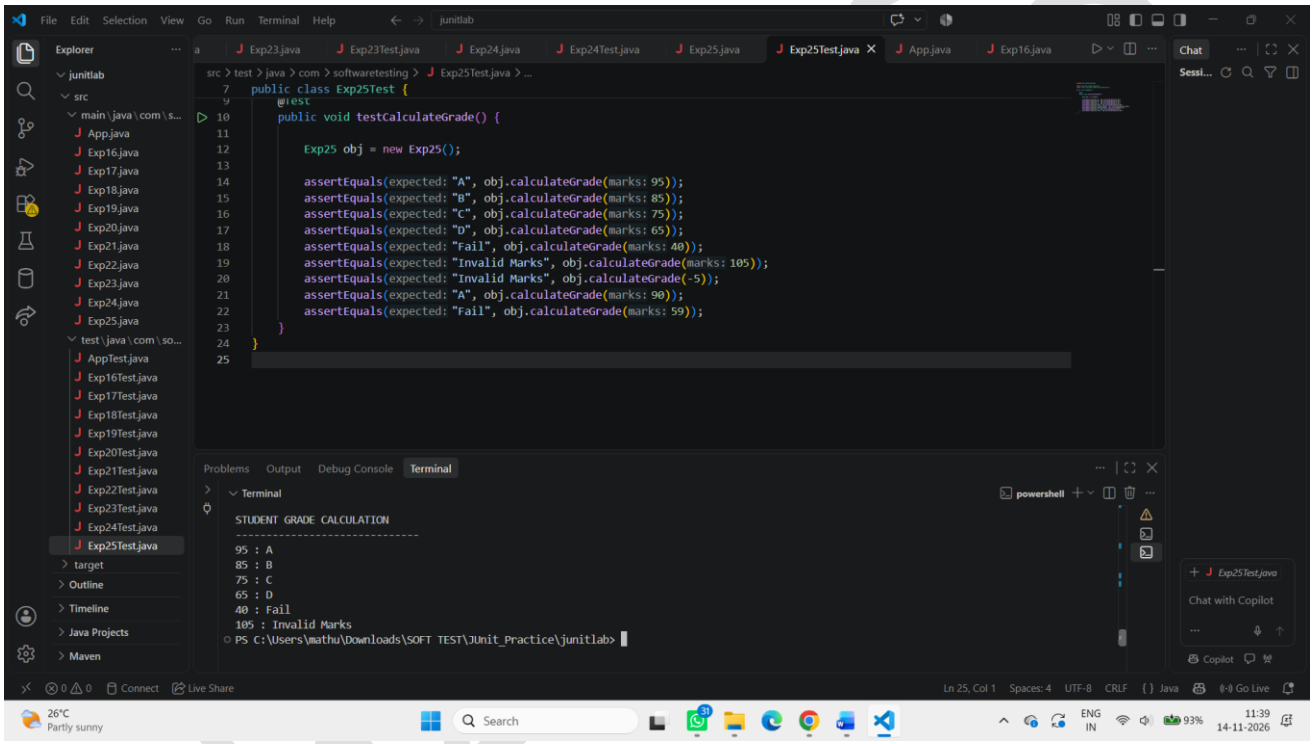
        Exp25 obj = new Exp25();

        assertEquals("A", obj.calculateGrade(95));
        assertEquals("B", obj.calculateGrade(85));
        assertEquals("C", obj.calculateGrade(75));
        assertEquals("D", obj.calculateGrade(65));
```

1931363

```
        assertEquals("Fail", obj.calculateGrade(40));
        assertEquals("Invalid Marks", obj.calculateGrade(105));
        assertEquals("Invalid Marks", obj.calculateGrade(-5));
        assertEquals("A", obj.calculateGrade(90));
        assertEquals("Fail", obj.calculateGrade(59));
    }
}
```

OUTPUT



RESULT

Thus, the student's grade was calculated and all grading conditions were verified successfully using JUnit.

26. Write a JUnit test case to find the largest element in an array and validate the result using white-box testing techniques.

AIM

To find the largest element in an integer array and verify the result using JUnit.

PROCEDURE

1. Check whether the array is null or empty.
2. Initialize the maximum with the first element.
3. Traverse the remaining elements and update the maximum when a larger value is found.
4. Test positive, negative, single-element, and empty arrays.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp26 {

    public int findLargest(int[] arr) {

        if (arr == null || arr.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }

        int largest = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > largest) {
                largest = arr[i];
            }
        }

        return largest;
    }
```

1923163

```
    public static void main(String[] args) {

        Exp26 obj = new Exp26();

        int[] arr1 = {10, 25, 15, 40, 30};
        int[] arr2 = {-5, -2, -10, -1};
        int[] arr3 = {7};

        System.out.println("LARGEST ELEMENT IN AN ARRAY");
        System.out.println("------------------------------");

        System.out.println("Array 1: 10, 25, 15, 40, 30");
        System.out.println("Largest: " + obj.findLargest(arr1));

        System.out.println("Array 2: -5, -2, -10, -1");
        System.out.println("Largest: " + obj.findLargest(arr2));

        System.out.println("Array 3: 7");
        System.out.println("Largest: " + obj.findLargest(arr3));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp26Test {

    @Test
    public void testFindLargest() {
```

192163

```
        Exp26 obj = new Exp26();

        assertEquals(40, obj.findLargest(
                new int[]{10, 25, 15, 40, 30}));

        assertEquals(-1, obj.findLargest(
                new int[]{-5, -2, -10, -1}));

        assertEquals(7, obj.findLargest(
                new int[]{7}));

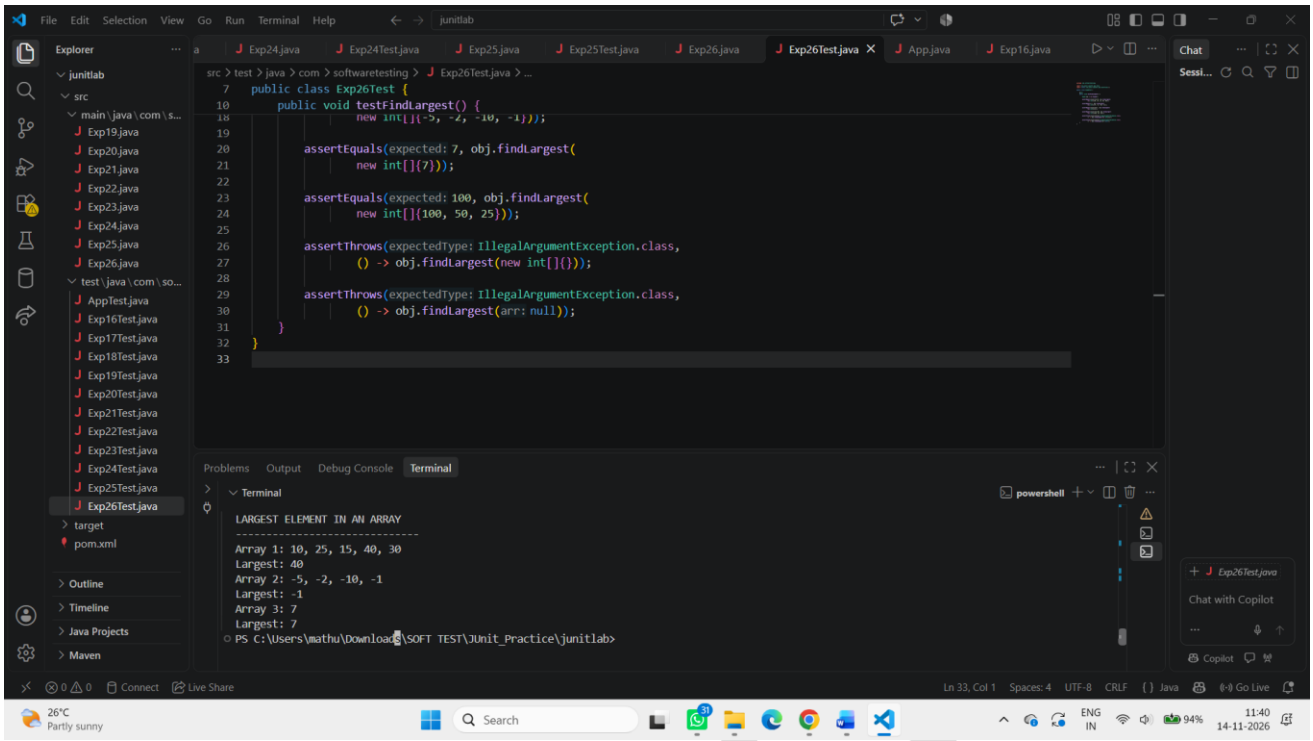
        assertEquals(100, obj.findLargest(
                new int[]{100, 50, 25}));

        assertThrows(IllegalArgumentException.class,
                () -> obj.findLargest(new int[]{}));

        assertThrows(IllegalArgumentException.class,
                () -> obj.findLargest(null));
    }
}
```

1923163

OUTPUT



RESULT

Thus, the largest array element was identified and validated successfully using JUnit.

192115

27. Write a program to calculate the average of n numbers using functions and verify the output using white-box testing.

AIM

To calculate the average of a set of numbers using a Java function and verify the result using JUnit.

PROCEDURE

1. Accept an array of numbers.
2. Check whether the array is null or empty.
3. Calculate the sum using a loop and divide by the number of elements.
4. Test integer, decimal, and empty-array inputs.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp27 {

    public double calculateAverage(int[] numbers) {

        if (numbers == null || numbers.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }

        int sum = 0;

        for (int number : numbers) {
            sum = sum + number;
        }

        return (double) sum / numbers.length;
    }

    public static void main(String[] args) {
```

1923163

```
        Exp27 obj = new Exp27();

        int[] arr1 = {10, 20, 30, 40, 50};
        int[] arr2 = {5, 10, 15};
        int[] arr3 = {100};

        System.out.println("AVERAGE OF N NUMBERS");
        System.out.println("------------------------------");

        System.out.println("Average of 10, 20, 30, 40, 50 : "
                + obj.calculateAverage(arr1));

        System.out.println("Average of 5, 10, 15 : "
                + obj.calculateAverage(arr2));

        System.out.println("Average of 100 : "
                + obj.calculateAverage(arr3));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp27Test {

    @Test
    public void testCalculateAverage() {

        Exp27 obj = new Exp27();
```

1923136

```
        assertEquals(30.0,
                obj.calculateAverage(new int[]{10, 20, 30, 40, 50}), 0.001);

        assertEquals(10.0,
                obj.calculateAverage(new int[]{5, 10, 15}), 0.001);

        assertEquals(100.0,
                obj.calculateAverage(new int[]{100}), 0.001);

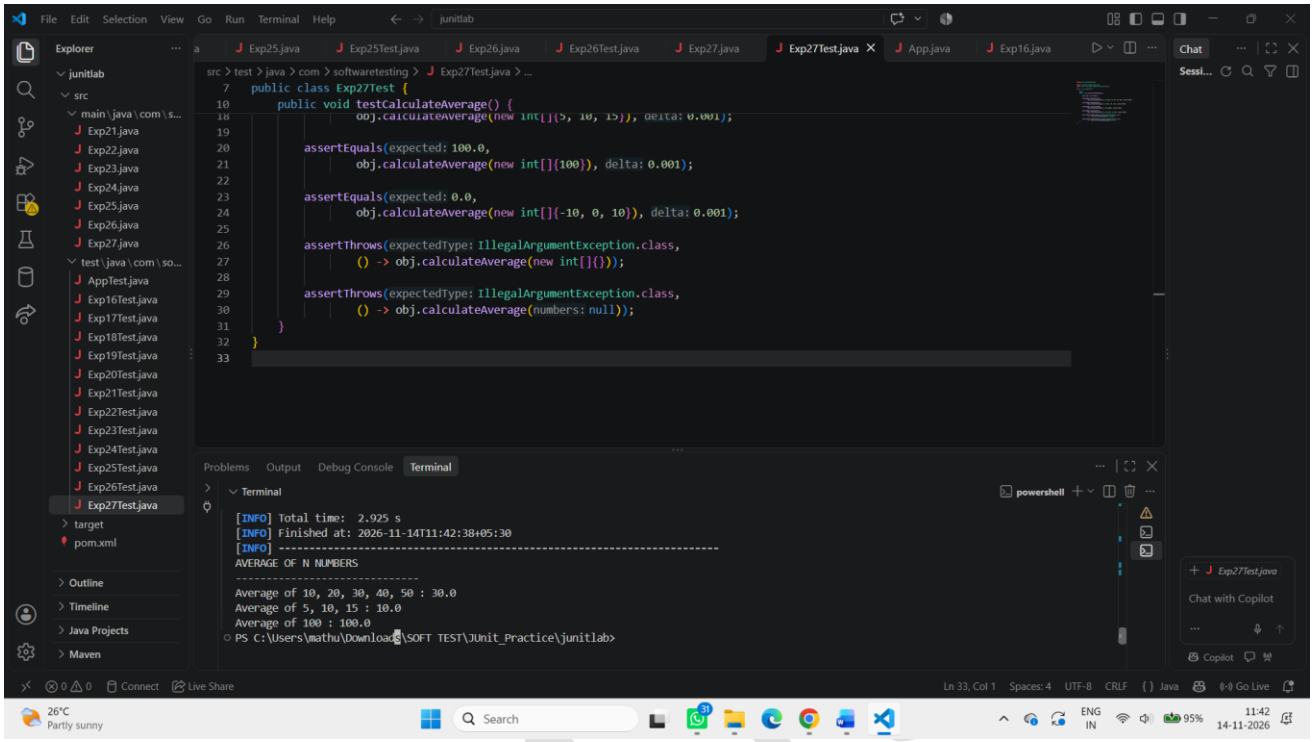
        assertEquals(0.0,
                obj.calculateAverage(new int[]{-10, 0, 10}), 0.001);

        assertThrows(IllegalArgumentException.class,
                () -> obj.calculateAverage(new int[]{}));

        assertThrows(IllegalArgumentException.class,
                () -> obj.calculateAverage(null));
    }
}
```

1923163

OUTPUT



RESULT

Thus, the average of the given numbers was calculated and verified successfully using JUnit.

19231

28. Write a JUnit program to check whether a given number is an Armstrong number and validate the output using assert statements.

AIM

To check whether a number is an Armstrong number and validate the result using JUnit.

PROCEDURE

1. Count the digits in the number.
2. Raise each digit to the power of the digit count and add the values.
3. Compare the sum with the original number.
4. Test Armstrong and non-Armstrong numbers.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp27 {

    public double calculateAverage(int[] numbers) {

        if (numbers == null || numbers.length == 0) {
            throw new IllegalArgumentException("Array cannot be empty");
        }

        int sum = 0;

        for (int number : numbers) {
            sum = sum + number;
        }

        return (double) sum / numbers.length;
    }

    public static void main(String[] args) {
```

1923163

```
        Exp27 obj = new Exp27();

        int[] arr1 = {10, 20, 30, 40, 50};
        int[] arr2 = {5, 10, 15};
        int[] arr3 = {100};

        System.out.println("AVERAGE OF N NUMBERS");
        System.out.println("------------------------------");

        System.out.println("Average of 10, 20, 30, 40, 50 : "
                + obj.calculateAverage(arr1));

        System.out.println("Average of 5, 10, 15 : "
                + obj.calculateAverage(arr2));

        System.out.println("Average of 100 : "
                + obj.calculateAverage(arr3));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp27Test {

    @Test
    public void testCalculateAverage() {

        Exp27 obj = new Exp27();
```

1923136

```
        assertEquals(30.0,
                obj.calculateAverage(new int[]{10, 20, 30, 40, 50}), 0.001);

        assertEquals(10.0,
                obj.calculateAverage(new int[]{5, 10, 15}), 0.001);

        assertEquals(100.0,
                obj.calculateAverage(new int[]{100}), 0.001);

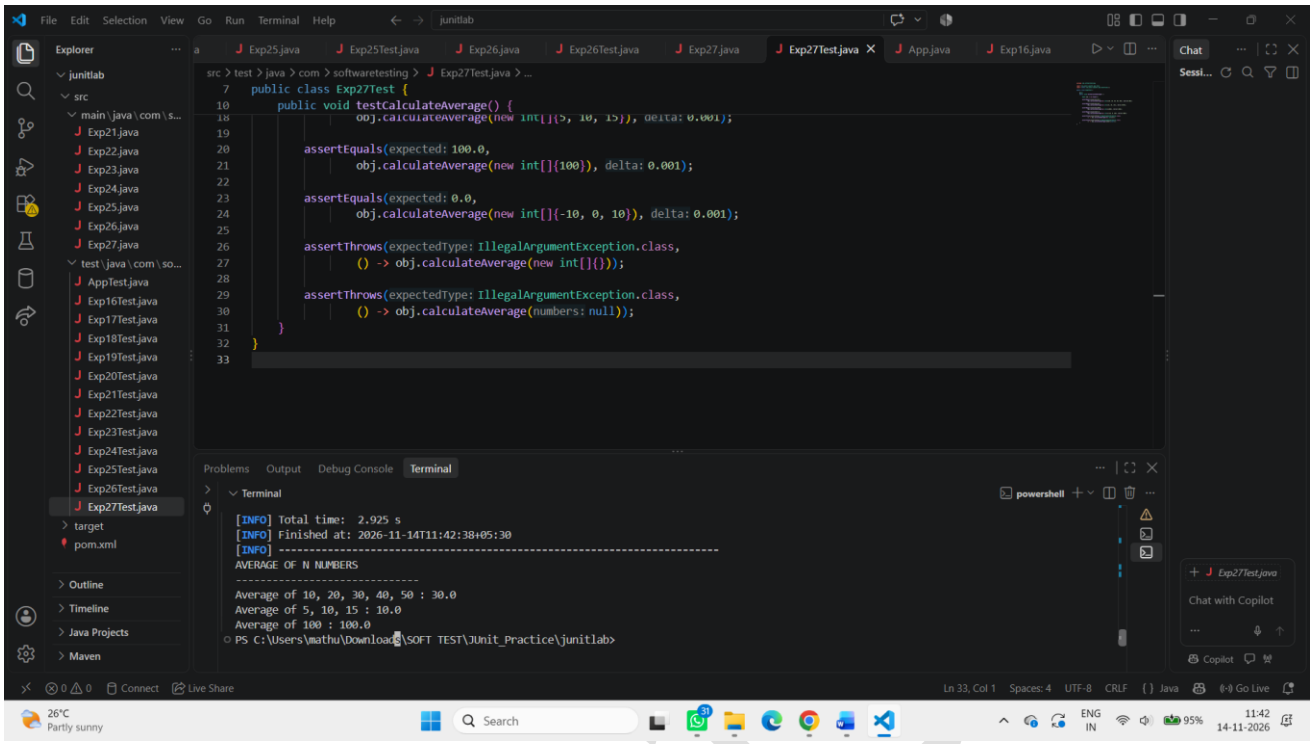
        assertEquals(0.0,
                obj.calculateAverage(new int[]{-10, 0, 10}), 0.001);

        assertThrows(IllegalArgumentException.class,
                () -> obj.calculateAverage(new int[]{}));

        assertThrows(IllegalArgumentException.class,
                () -> obj.calculateAverage(null));
    }
}
```

1923163

OUTPUT



RESULT

Thus, Armstrong and non-Armstrong numbers were identified and verified successfully using JUnit.

192115

29. Write a program to convert temperature from Celsius to Fahrenheit. Verify the output values using white-box testing.

AIM

To convert temperature from Celsius to Fahrenheit using a Java function and verify the result using JUnit.

PROCEDURE

1. Accept temperature in Celsius.
2. Apply Fahrenheit = (Celsius × 9/5) + 32.
3. Return the converted temperature.
4. Test zero, boiling point, and negative temperature values.

PROGRAM

Implementation file:

```
package com.softwaretesting;

public class Exp28 {

   public boolean isArmstrong(int number) {

      if (number < 0) {
         return false;
      }

      int original = number;
      int sum = 0;
      int digits = String.valueOf(number).length();

      while (number != 0) {
         int digit = number % 10;
         sum += Math.pow(digit, digits);
         number = number / 10;
      }
```

1923163

```
        return sum == original;
    }

    public static void main(String[] args) {

        Exp28 obj = new Exp28();

        System.out.println("ARMSTRONG NUMBER CHECKING");
        System.out.println("-----------------------------");

        System.out.println("153 : " +
                (obj.isArmstrong(153) ? "Armstrong" : "Not Armstrong"));

        System.out.println("370 : " +
                (obj.isArmstrong(370) ? "Armstrong" : "Not Armstrong"));

        System.out.println("123 : " +
                (obj.isArmstrong(123) ? "Armstrong" : "Not Armstrong"));

        System.out.println("9474 : " +
                (obj.isArmstrong(9474) ? "Armstrong" : "Not Armstrong"));

        System.out.println("-153 : " +
                (obj.isArmstrong(-153) ? "Armstrong" : "Not Armstrong"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

1931363

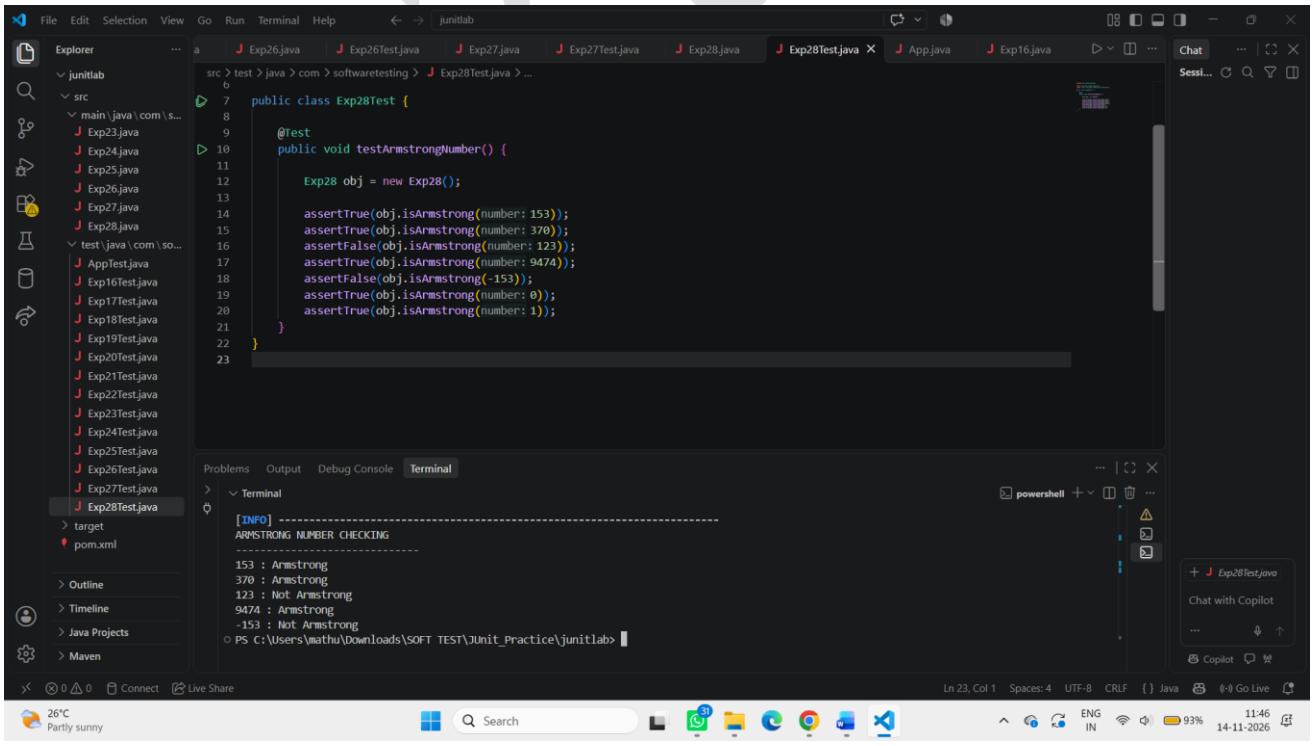
```
public class Exp28Test {

    @Test
    public void testArmstrongNumber() {

        Exp28 obj = new Exp28();

        assertTrue(obj.isArmstrong(153));
        assertTrue(obj.isArmstrong(370));
        assertFalse(obj.isArmstrong(123));
        assertTrue(obj.isArmstrong(9474));
        assertFalse(obj.isArmstrong(-153));
        assertTrue(obj.isArmstrong(0));
        assertTrue(obj.isArmstrong(1));
    }
}
```

OUTPUT



RESULT

Thus, Celsius temperatures were converted into Fahrenheit and verified successfully using JUnit.

30. Write a JUnit test case to find the length of a string without using built-in functions and validate the output using white-box testing techniques.

AIM

To find the length of a string without using the built-in length() method and verify the result using JUnit.

PROCEDURE

1. Convert the string to a character array.
2. Initialize a counter to zero.
3. Traverse the array and increment the counter for every character.
4. Test normal, empty, and null inputs.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp29 {


   public double convertToFahrenheit(double celsius) {
      return (celsius * 9 / 5) + 32;
   }


   public static void main(String[] args) {


      Exp29 obj = new Exp29();


      System.out.println("CELSIUS TO FAHRENHEIT CONVERSION");
      System.out.println("--------------------------------");


      System.out.println("0 Celsius = " + obj.convertToFahrenheit(0) + " Fahrenheit");
      System.out.println("25 Celsius = " + obj.convertToFahrenheit(25) + " Fahrenheit");
      System.out.println("100 Celsius = " + obj.convertToFahrenheit(100) + " Fahrenheit");
      System.out.println("-40 Celsius = " + obj.convertToFahrenheit(-40) + " Fahrenheit");
      System.out.println("37 Celsius = " + obj.convertToFahrenheit(37) + " Fahrenheit");
   }
```

1931363

```
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp29Test {

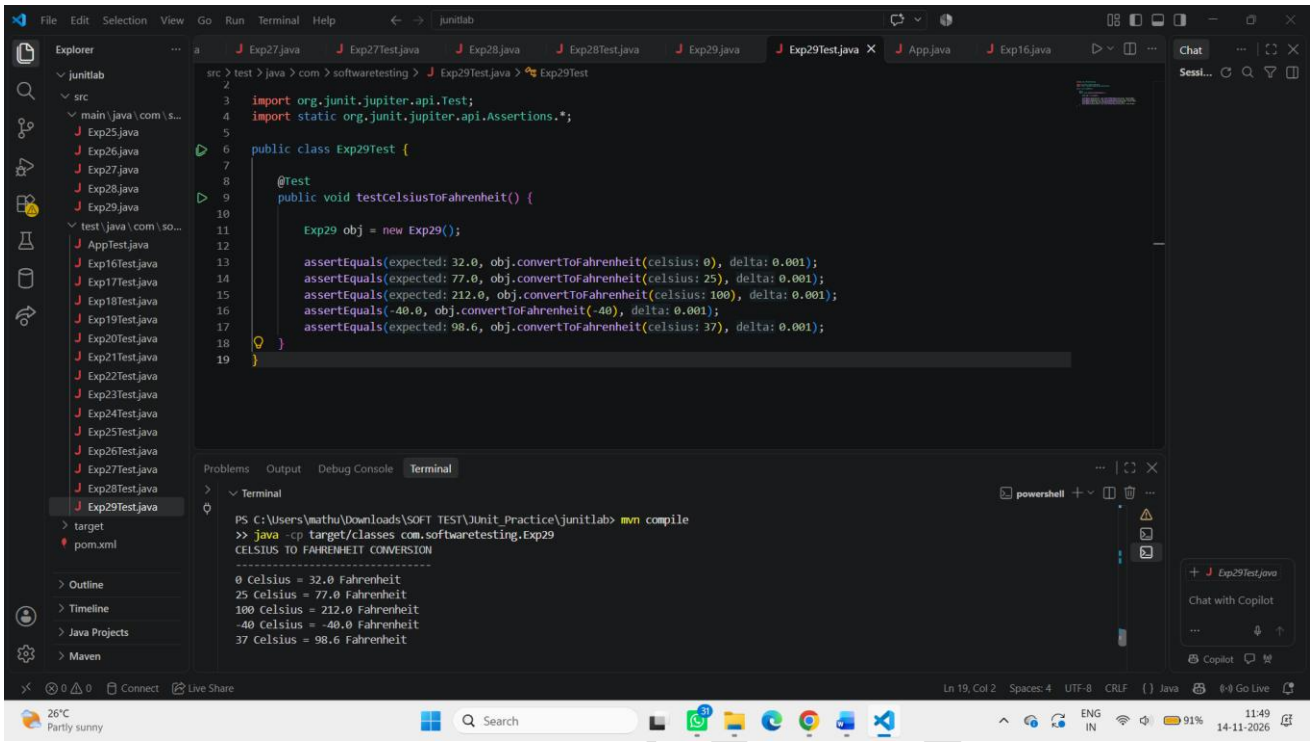
   @Test
   public void testCelsiusToFahrenheit() {

      Exp29 obj = new Exp29();

      assertEquals(32.0, obj.convertToFahrenheit(0), 0.001);
      assertEquals(77.0, obj.convertToFahrenheit(25), 0.001);
      assertEquals(212.0, obj.convertToFahrenheit(100), 0.001);
      assertEquals(-40.0, obj.convertToFahrenheit(-40), 0.001);
      assertEquals(98.6, obj.convertToFahrenheit(37), 0.001);
   }
}
```

192113363

OUTPUT



RESULT

Thus, the string length was calculated without using length() and verified successfully using JUnit.

192315

31. Demonstrate the working of JUnit to reverse a word and use assert statements to validate the output.

AIM

To reverse a word using Java and validate the reversed word using JUnit assertions.

PROCEDURE

1. Traverse the word from the last character to the first.
2. Append each character to a StringBuilder.
3. Return the reversed word.
4. Test normal, single-character, and empty words.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp30 {


   public int findLength(String str) {


      int count = 0;


      for (char ch : str.toCharArray()) {
         count++;
      }


      return count;
   }


   public static void main(String[] args) {


      Exp30 obj = new Exp30();


      System.out.println("STRING LENGTH WITHOUT BUILT-IN FUNCTION");
      System.out.println("---------------------------------------");
```

1931363

```
        System.out.println("Hello : " + obj.findLength("Hello"));
        System.out.println("Java : " + obj.findLength("Java"));
        System.out.println("Testing : " + obj.findLength("Testing"));
        System.out.println("A : " + obj.findLength("A"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp30Test {

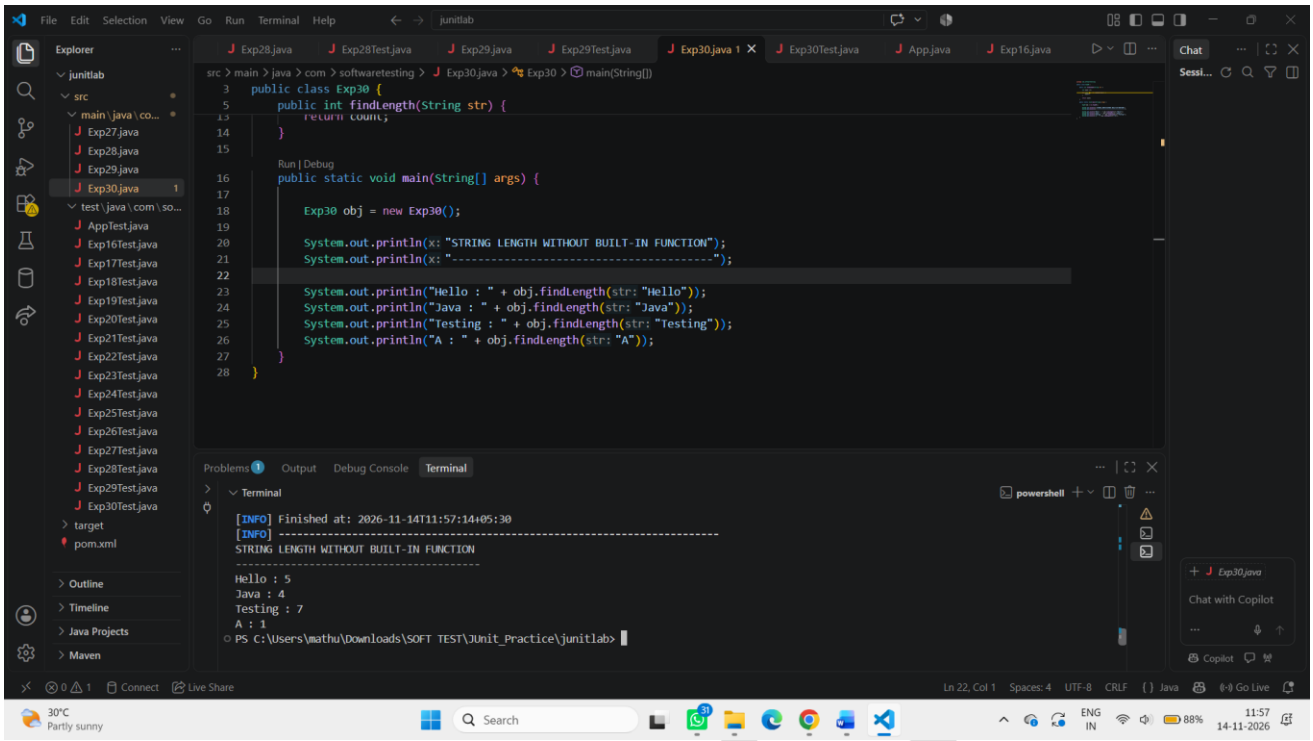
    @Test
    public void testStringLength() {

        Exp30 obj = new Exp30();

        assertEquals(5, obj.findLength("Hello"));
        assertEquals(4, obj.findLength("Java"));
        assertEquals(7, obj.findLength("Testing"));
        assertEquals(1, obj.findLength("A"));
        assertEquals(0, obj.findLength(""));
    }
}
```

1923131363

OUTPUT



RESULT

Thus, the given word was reversed and the output was validated successfully using JUnit.

192115

32. Write a white-box test case (JUnit) to perform string comparison and use assert statements to validate the output.

AIM

To compare two strings lexicographically and validate the result using JUnit assertions.

PROCEDURE

1. Create a function that compares two strings using compareTo().
2. Return the integer comparison result.
3. Test equal strings, a string that comes before another, and one that comes after another.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp31 {


   public boolean isPalindrome(String str) {


      String reverse = "";


      for (int i = str.length() - 1; i >= 0; i--) {
         reverse = reverse + str.charAt(i);
      }


      return str.equalsIgnoreCase(reverse);
   }


   public static void main(String[] args) {


      Exp31 obj = new Exp31();


      System.out.println("STRING PALINDROME CHECKING");
      System.out.println("--------------------------");


      System.out.println("MADAM : " + obj.isPalindrome("MADAM"));
```

1931363

```
        System.out.println("LEVEL : " + obj.isPalindrome("LEVEL"));
        System.out.println("HELLO : " + obj.isPalindrome("HELLO"));
        System.out.println("RADAR : " + obj.isPalindrome("RADAR"));
        System.out.println("JAVA : " + obj.isPalindrome("JAVA"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp31Test {

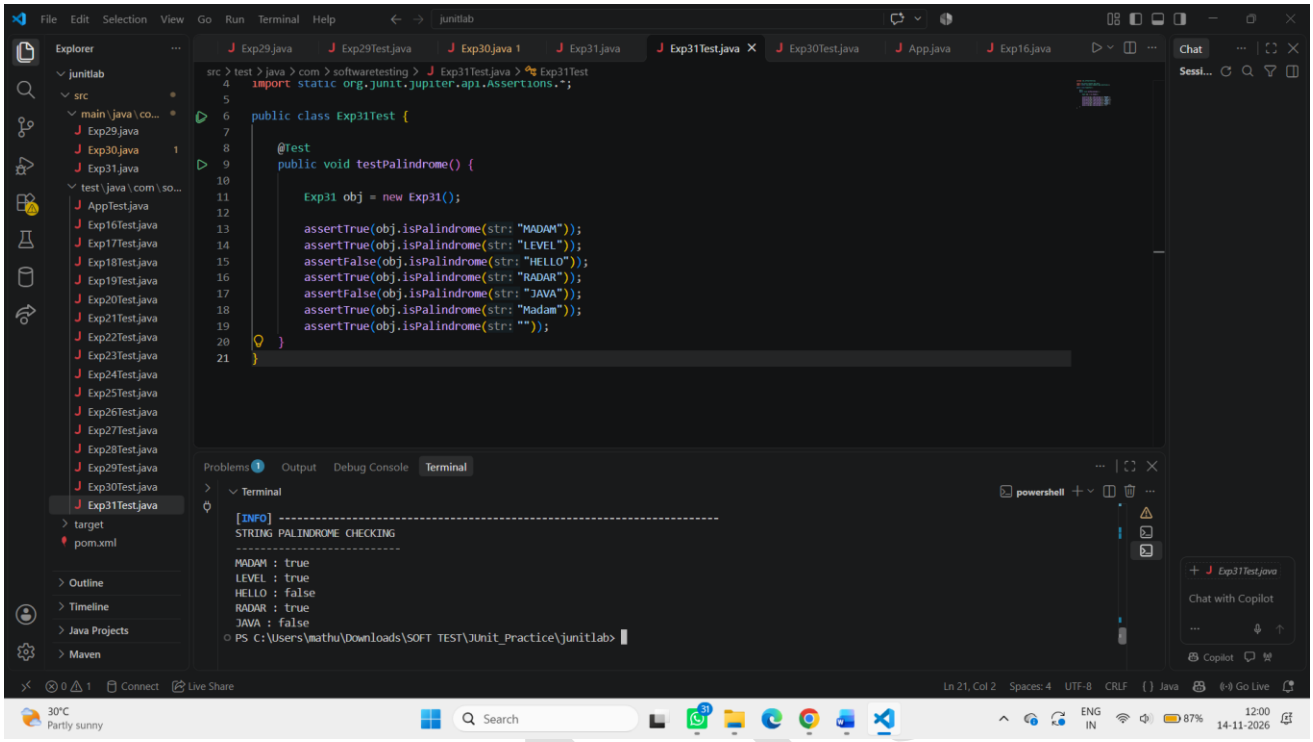
    @Test
    public void testPalindrome() {

        Exp31 obj = new Exp31();

        assertTrue(obj.isPalindrome("MADAM"));
        assertTrue(obj.isPalindrome("LEVEL"));
        assertFalse(obj.isPalindrome("HELLO"));
        assertTrue(obj.isPalindrome("RADAR"));
        assertFalse(obj.isPalindrome("JAVA"));
        assertTrue(obj.isPalindrome("Madam"));
        assertTrue(obj.isPalindrome(""));
    }
}
```

1923131363

OUTPUT



RESULT

Thus, string comparison was performed and the results were validated successfully using JUnit.

192317

33. Write a JUnit test case for a voting system, use assert statements, and validate it using white-box testing techniques.

AIM

To check voter eligibility based on age and validate the decision using JUnit assertions.

PROCEDURE

1. Accept the voter's age as input.
2. Return true if the age is 18 or above; otherwise return false.
3. Test ages below 18, exactly 18, and above 18.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp32 {


   public boolean isPerfect(int number) {


      if (number <= 1) {
         return false;
      }


      int sum = 0;


      for (int i = 1; i <= number / 2; i++) {
         if (number % i == 0) {
            sum += i;
         }
      }


      return sum == number;
   }


   public static void main(String[] args) {
```

1923136

```
        Exp32 obj = new Exp32();

        System.out.println("PERFECT NUMBER CHECKING");
        System.out.println("----------------------");

        System.out.println("6 : " + (obj.isPerfect(6) ? "Perfect Number" : "Not Perfect Number"));
        System.out.println("28 : " + (obj.isPerfect(28) ? "Perfect Number" : "Not Perfect Number"));
        System.out.println("12 : " + (obj.isPerfect(12) ? "Perfect Number" : "Not Perfect Number"));
        System.out.println("496 : " + (obj.isPerfect(496) ? "Perfect Number" : "Not Perfect Number"));
        System.out.println("1 : " + (obj.isPerfect(1) ? "Perfect Number" : "Not Perfect Number"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

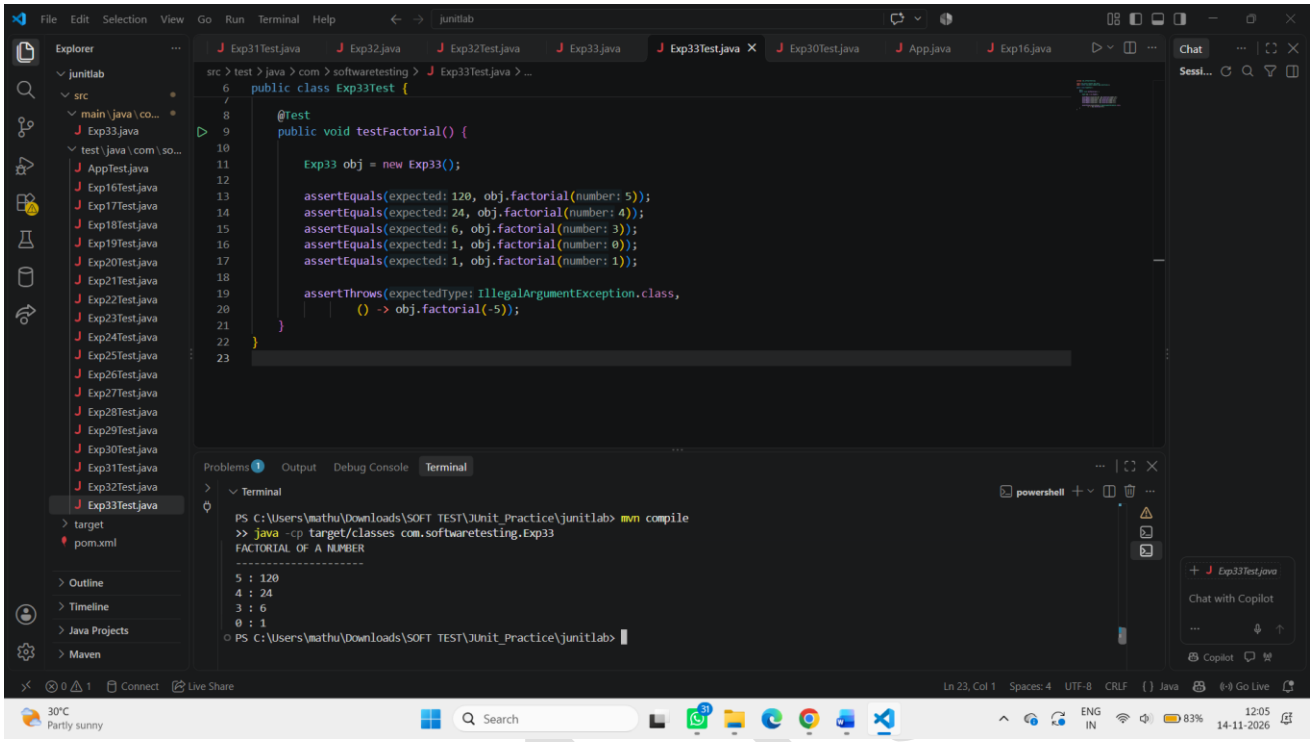
public class Exp32Test {

    @Test
    public void testPerfectNumber() {

        Exp32 obj = new Exp32();
        assertTrue(obj.isPerfect(6));
        assertTrue(obj.isPerfect(28));
        assertFalse(obj.isPerfect(12));
        assertTrue(obj.isPerfect(496));
        assertFalse(obj.isPerfect(1));
        assertFalse(obj.isPerfect(0));
        assertFalse(obj.isPerfect(-6));
    }
}
```

192113063

OUTPUT



RESULT

Thus, voter eligibility was checked based on age and validated successfully using JUnit.

192317

34. Write a program using functions to calculate Simple Interest. Suppose the customer is a senior citizen. He is offered 12 percent rate of interest; for all other customers, the rate of interest is 10 percent. The output values should be verified using white-box testing.

AIM

To calculate simple interest at 12% for senior citizens and 10% for other customers and verify the results using JUnit.

PROCEDURE

1. Accept principal, time, and senior-citizen status.
2. Select 12% for senior citizens and 10% for other customers.
3. Calculate SI = (Principal × Rate × Time) / 100.
4. Test both customer categories using JUnit assertions.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp34 {


   public int calculateSum(int number) {


      if (number < 0) {
         throw new IllegalArgumentException("Negative numbers are not allowed");
      }


      int sum = 0;


      for (int i = 1; i <= number; i++) {
         sum += i;
      }


      return sum;
   }


   public static void main(String[] args) {
```

1931363

```
        Exp34 obj = new Exp34();

        System.out.println("SUM OF NATURAL NUMBERS");
        System.out.println("----------------------");

        System.out.println("5 : " + obj.calculateSum(5));
        System.out.println("10 : " + obj.calculateSum(10));
        System.out.println("15 : " + obj.calculateSum(15));
        System.out.println("0 : " + obj.calculateSum(0));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp34Test {

    @Test
    public void testSumOfNaturalNumbers() {

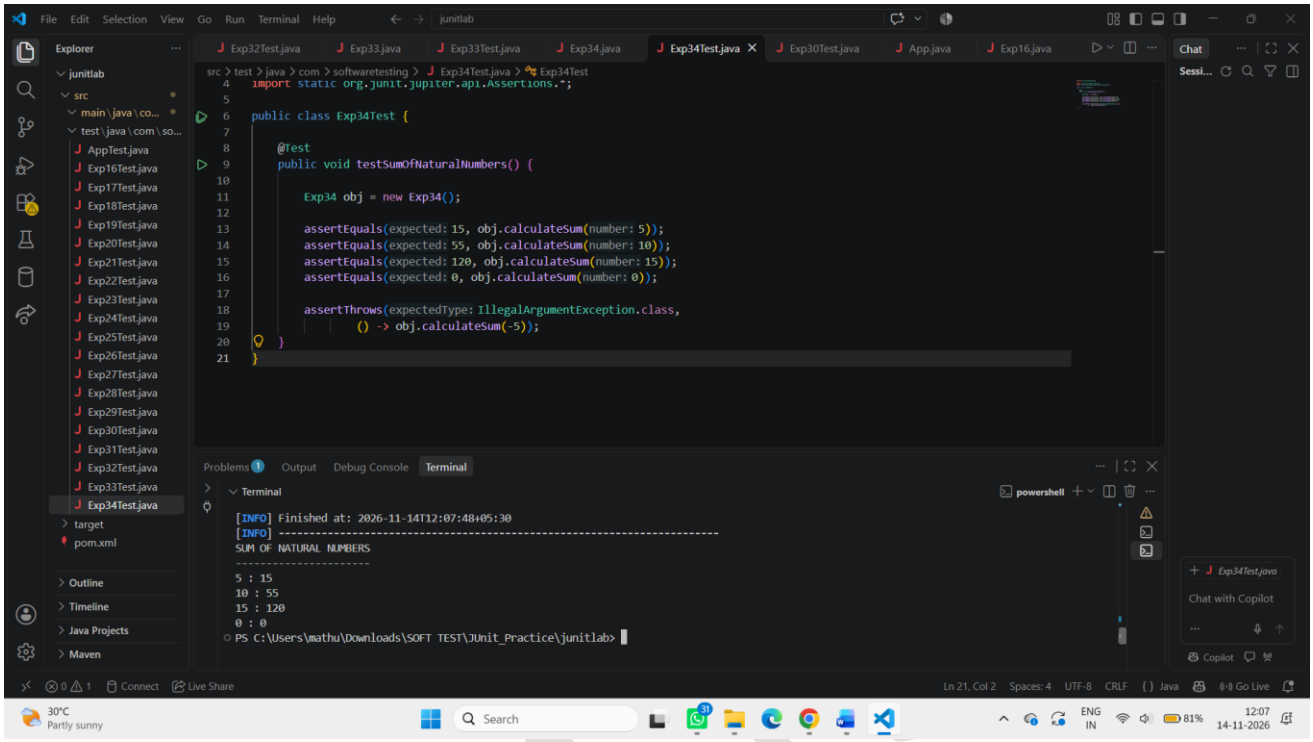
        Exp34 obj = new Exp34();

        assertEquals(15, obj.calculateSum(5));
        assertEquals(55, obj.calculateSum(10));
        assertEquals(120, obj.calculateSum(15));
        assertEquals(0, obj.calculateSum(0));

        assertThrows(IllegalArgumentException.class,
                () -> obj.calculateSum(-5));
    }
}
```

1923163

OUTPUT



RESULT

Thus, simple interest was calculated using the correct rate for each customer category and verified successfully using JUnit.

19231

35. Check whether the given number is a palindrome or not, and verify the output values using white-box testing.

AIM

To check whether an integer is a palindrome and verify the result using JUnit assertions.

PROCEDURE

1. Store the original number.
2. Reverse the digits using modulus and division operations.
3. Compare the reversed number with the original number.
4. Test palindrome, non-palindrome, zero, and negative values.

PROGRAM

Implementation file:

```
package com.softwaretesting;


public class Exp35 {


   public boolean isLeapYear(int year) {


      if (year <= 0) {
         return false;
      }


      if (year % 400 == 0) {
         return true;
      } else if (year % 100 == 0) {
         return false;
      } else if (year % 4 == 0) {
         return true;
      } else {
         return false;
      }
   }


   public static void main(String[] args) {
```

1923163

```
        Exp35 obj = new Exp35();

        System.out.println("LEAP YEAR CHECKING");
        System.out.println("------------------");

        System.out.println("2024 : " + (obj.isLeapYear(2024) ? "Leap Year" : "Not a Leap Year"));
        System.out.println("2023 : " + (obj.isLeapYear(2023) ? "Leap Year" : "Not a Leap Year"));
        System.out.println("2000 : " + (obj.isLeapYear(2000) ? "Leap Year" : "Not a Leap Year"));
        System.out.println("1900 : " + (obj.isLeapYear(1900) ? "Leap Year" : "Not a Leap Year"));
        System.out.println("2028 : " + (obj.isLeapYear(2028) ? "Leap Year" : "Not a Leap Year"));
    }
}
```

JUnit test file:

```
package com.softwaretesting;

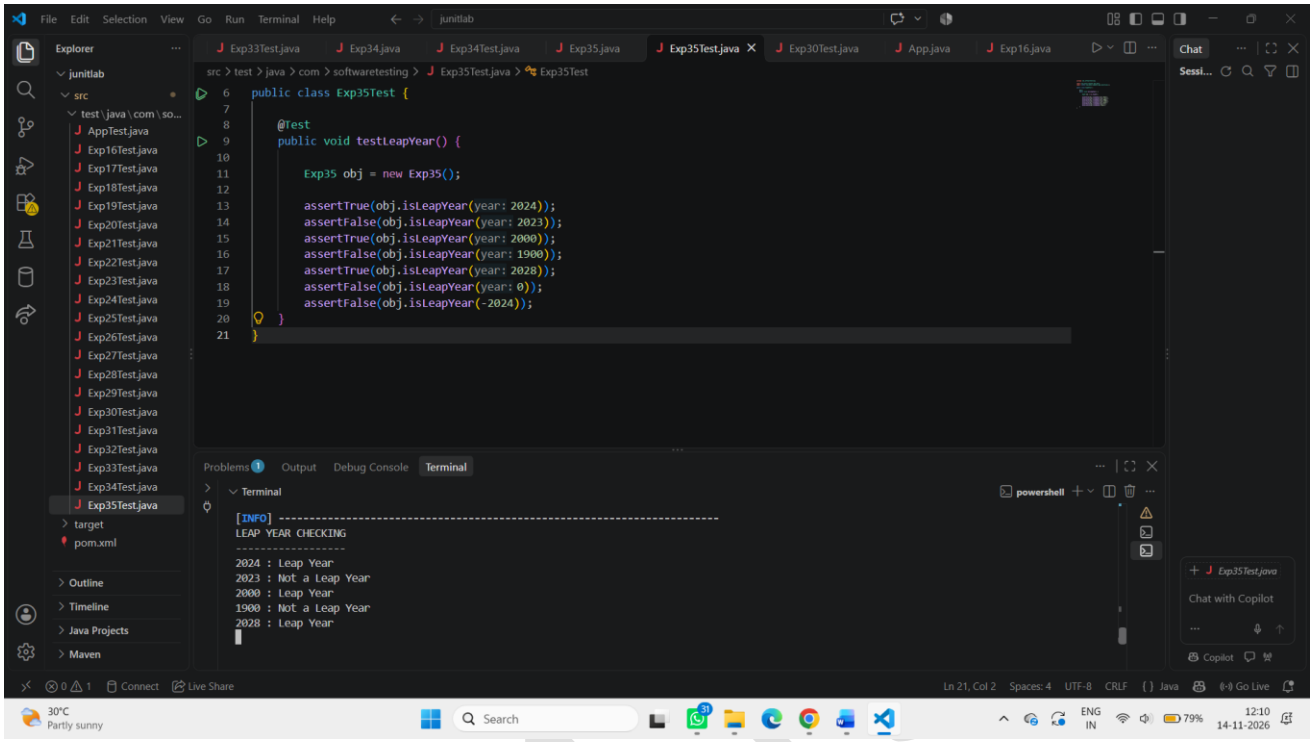
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Exp35Test {

    @Test
    public void testLeapYear() {
        Exp35 obj = new Exp35();
        assertTrue(obj.isLeapYear(2024));
        assertFalse(obj.isLeapYear(2023));
        assertTrue(obj.isLeapYear(2000));
        assertFalse(obj.isLeapYear(1900));
        assertTrue(obj.isLeapYear(2028));
        assertFalse(obj.isLeapYear(0));
        assertFalse(obj.isLeapYear(-2024));
    }
}
```

1931363

OUTPUT



RESULT

Thus, palindrome and non-palindrome numbers were identified and verified successfully using JUnit.

192211